

一、技术选型与可行性分析

1.1 核心库：modern-xlsx

经过调研，**modern-xlsx** 是目前前端 Excel 导出领域最适合的 WASM 方案，核心优势如下：

性能数据（来自官方 benchmark，Node.js 单线程 v1.0.0）：

操作	modern-xlsx	SheetJS CE	提升倍数
写入 10 万行（batch aoaToSheet）	232 ms	1,950 ms	8.4x
写入 5 万行（batch aoaToSheet）	49 ms	80 ms	1.6x
读取 10 万行	472 ms	1,901 ms	4.0x
读取 1 万行	69 ms	170 ms	2.5x

功能对比（与 SheetJS）：

功能	modern-xlsx	SheetJS CE	SheetJS Pro
单元格样式（字体/填充/边框）	✅ 免费	❌	💰 付费
数字格式	✅ 免费	只读	💰 付费
数据验证	✅ 免费	❌	💰 付费
条件格式	✅ 免费	❌	💰 付费
冻结窗格	✅ 免费	部分	💰 付费
单元格注释	✅ 免费	只读	💰 付费
工作表保护/加密	✅ 免费	只读	💰 付费

包体积：ESM 133KB + WASM 1.1MB，零运行时依赖。

1.2 备选方案对比

方案	优势	劣势	适用场景
----	----	----	------

modern-xlsx	功能最全、性能最优、API 设计现代化	WASM 体积 1.1MB	首选方案
wasm-xlsxwriter	轻量级、API 简洁	功能相对较少	简单导出场景
turbo-xlsx	流式写入、加密支持	生态较新	超大文件场景
@senlinz/import-export	开箱即用、API 友好	定制化受限	快速接入场景

结论：modern-xlsx 功能最全面、性能最优，且开源免费，是承载 20% 挑战目标的最佳选择。

二、Monorepo 共享包架构

2.1 目录结构

```
packages/
├─ excel-exporter/                # 新增：Excel 导出核心包
│  ├─ src/
│  │   ├─ index.ts                # 对外暴露的统一 API
│  │   ├─ wasm-loader.ts          # WASM 模块加载与生命周期管理
│  │   ├─ workbook-builder.ts      # 工作簿构建器（核心逻辑）
│  │   ├─ style-presets.ts         # 预设样式模板库
│  │   ├─ types.ts                 # 完整 TypeScript 类型定义
│  │   └─ workers/
│  │       └─ export.worker.ts     # Web Worker 导出线程
│  │       └─ __tests__/
│  │           ├─ performance.test.ts # 性能基准测试
│  │           └─ integration.test.ts  # 集成测试
│  └─ package.json
│  └─ tsconfig.json
│  └─ README.md
├─ apps/
│  ├─ admin-a/                    # 消费方：管理后台 A
│  └─ admin-b/                    # 消费方：管理后台 B
```

2.2 package.json 配置

```
{
  "name": "@your-org/excel-exporter",
  "version": "0.1.0",
  "type": "module",
  "main": "./dist/index.js",
  "types": "./dist/index.d.ts",
  "exports": {
    ".": {
      "types": "./dist/index.d.ts",
      "import": "./dist/index.js"
    },
    "./styles": {
      "types": "./dist/style-presets.d.ts",
      "import": "./dist/style-presets.js"
    }
  },
  "peerDependencies": {
    "modern-xlsx": "^1.2.0"
  },
  "files": ["dist"]
}
```

2.3 各应用引用方式

```
// apps/admin-a/package.json
{
  "dependencies": {
    "@your-org/excel-exporter": "workspace:*"
  }
}
```

三、核心模块详细设计

3.1 WASM 加载器 (wasm-loader.ts)

设计要点：

- 懒加载：仅在首次调用导出时加载 WASM，不影响首屏性能
- 单例模式：全局仅初始化一次
- 加载状态管理：支持加载中、成功、失败三种状态
- 超时与重试：加载超时自动重试，最多 3 次

```

// packages/excel-exporter/src/wasm-loader.ts
import { initWasm } from 'modern-xlsx';

type LoadState = 'idle' | 'loading' | 'ready' | 'error';

class WasmLoader {
  private state: LoadState = 'idle';
  private initPromise: Promise<void> | null = null;

  async ensureLoaded(): Promise<void> {
    if (this.state === 'ready') return;
    if (this.state === 'loading' && this.initPromise) {
      return this.initPromise;
    }

    this.state = 'loading';
    this.initPromise = this.doLoad();
    return this.initPromise;
  }

  private async doLoad(): Promise<void> {
    try {
      // 设置超时 (10秒)
      const timeoutPromise = new Promise((_, reject) => {
        setTimeout(() => reject(new Error('WASM 加载超时')), 10000);
      });

      await Promise.race([initWasm(), timeoutPromise]);
      this.state = 'ready';
    } catch (error) {
      this.state = 'error';
      throw new Error(`WASM 加载失败: ${error.message}`);
    }
  }

  get isReady(): boolean { return this.state === 'ready'; }
}

```

```
export const wasmLoader = new WasmLoader();
```

3.2 类型定义 (types.ts)

```
// packages/excel-exporter/src/types.ts

/** 单元格样式配置 */
export interface CellStyle {
  font?: {
    bold?: boolean;
    size?: number;
    color?: string;
    name?: string;
  };
  fill?: {
    pattern?: 'solid';
    fgColor?: string;
  };
  alignment?: {
    horizontal?: 'left' | 'center' | 'right';
    vertical?: 'top' | 'center' | 'bottom';
    wrapText?: boolean;
  };
  border?: {
    top?: { style: 'thin' | 'medium' | 'thick'; color: string };
    bottom?: { style: 'thin' | 'medium' | 'thick'; color: string };
    left?: { style: 'thin' | 'medium' | 'thick'; color: string };
    right?: { style: 'thin' | 'medium' | 'thick'; color: string };
  };
  numFormat?: string;
}

/** 列配置 */
export interface ColumnConfig {
  key: string;
  header: string;
  width?: number;
  style?: CellStyle;
  /** 自定义单元格格式化函数 */
  format?: (value: any) => string | number;
}
```

```
/** Sheet 配置 */
export interface SheetConfig {
  name: string;
  columns: ColumnConfig[];
  data: Record<string, any>[];
  /** 冻结行数（通常冻结表头） */
  freezeRows?: number;
  /** 合并单元格配置 */
  merges?: Array<{
    startRow: number;
    endRow: number;
    startCol: number;
    endCol: number;
  }>;
}

/** 导出选项 */
export interface ExportOptions {
  sheets: SheetConfig[];
  filename: string;
  /** 是否使用 Worker 线程（大数据量时自动启用） */
  useWorker?: boolean;
  /** 导出进度回调 */
  onProgress?: (progress: number) => void;
}

/** 导出结果 */
export interface ExportResult {
  success: boolean;
  blob?: Blob;
  error?: Error;
  /** 实际耗时 (ms) */
  duration?: number;
  /** 数据行数 */
  rowCount?: number;
}
```


3.3 工作簿构建器 (workbook-builder.ts)

```
// packages/excel-exporter/src/workbook-builder.ts
import { Workbook, type Style } from 'modern-xlsx';
import type { SheetConfig, ExportOptions, CellStyle } from './types';
import { wasmLoader } from './wasm-loader';
import { applyStyle } from './style-utils';

export class WorkbookBuilder {
  private workbook: Workbook;

  private constructor() {
    this.workbook = new Workbook();
  }

  static async create(): Promise<WorkbookBuilder> {
    await wasmLoader.ensureLoaded();
    return new WorkbookBuilder();
  }

  /** 添加 Sheet */
  addSheet(config: SheetConfig): this {
    const ws = this.workbook.addSheet(config.name);

    // 1. 写入表头
    config.columns.forEach((col, colIndex) => {
      ws.cell(0, colIndex).value = col.header;
      if (col.style) {
        const styleIndex = applyStyle(this.workbook, col.style);
        ws.cell(0, colIndex).styleIndex = styleIndex;
      }
      if (col.width) {
        ws.setColumnWidth(colIndex, col.width);
      }
    });

    // 2. 批量写入数据 (利用 batch API 获得最优性能)
    const rowData = config.data.map((item) =>
      config.columns.map((col) => {
```

```

        const raw = item[col.key];
        return col.format ? col.format(raw) : raw;
    })
);

// 批量写入：使用 aoaToSheet 方式，性能最优
ws.batch().writeRows(1, rowData);

// 3. 冻结窗格
if (config.freezeRows && config.freezeRows > 0) {
    ws.setFreezePanels(config.freezeRows, 0);
}

// 4. 合并单元格
config.merges?.forEach((merge) => {
    ws.mergeRange(
        merge.startRow,
        merge.startCol,
        merge.endRow,
        merge.endCol,
        ''
    );
});

return this;
}

/** 导出为 Blob */
async toBlob(): Promise<Blob> {
    const buffer = await this.workbook.toBuffer();
    return new Blob([buffer], {
        type: 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'
    });
}

/** 导出并下载 */

```

```
async download(filename: string): Promise<void> {
  const blob = await this.toBlob();
  const url = URL.createObjectURL(blob);
  const link = document.createElement('a');
  link.href = url;
  link.download = filename.endsWith('.xlsx') ? filename :
`${filename}.xlsx`;
  document.body.appendChild(link);
  link.click();
  document.body.removeChild(link);
  URL.revokeObjectURL(url);
}
}
```

3.4 样式工具 (style-utils.ts)

```
// packages/excel-exporter/src/style-utils.ts
import { Workbook, type Style } from 'modern-xlsx';
import type { CellStyle } from './types';

export function applyStyle(wb: Workbook, style: CellStyle): number {
  let s = wb.createStyle();

  if (style.font) {
    if (style.font.bold !== undefined) s = s.font({ bold:
style.font.bold });
    if (style.font.size) s = s.font({ size: style.font.size });
    if (style.font.color) s = s.font({ color: style.font.color });
    if (style.font.name) s = s.font({ name: style.font.name });
  }

  if (style.fill?.fgColor) {
    s = s.fill({
      pattern: style.fill.pattern || 'solid',
      fgColor: style.fill.fgColor
    });
  }

  if (style.alignment) {
    s = s.alignment({
      horizontal: style.alignment.horizontal || 'left',
      vertical: style.alignment.vertical || 'center',
      wrapText: style.alignment.wrapText || false
    });
  }

  if (style.numFormat) {
    s = s.setNumFormat(style.numFormat);
  }

  // 边框处理 (modern-xlsx 支持链式调用)
  if (style.border) {
    // ... 边框配置
  }
}
```

```
}  
  
    return s.build(wb.styles);  
}
```

3.5 预设样式模板 (style-presets.ts)

```
// packages/excel-exporter/src/style-presets.ts
import type { CellStyle } from './types';

export const StylePresets = {
  /** 表头样式：加粗、深蓝背景、白色文字、居中 */
  header: {
    font: { bold: true, size: 12, color: 'FFFFFF' },
    fill: { pattern: 'solid', fgColor: '1F4E79' },
    alignment: { horizontal: 'center', vertical: 'center' }
  } as CellStyle,

  /** 金额样式：保留两位小数、千分位 */
  currency: {
    numFormat: '#,##0.00',
    alignment: { horizontal: 'right' }
  } as CellStyle,

  /** 百分比样式 */
  percent: {
    numFormat: '0.00%',
    alignment: { horizontal: 'right' }
  } as CellStyle,

  /** 日期样式：YYYY-MM-DD */
  date: {
    numFormat: 'yyyy-mm-dd',
    alignment: { horizontal: 'center' }
  } as CellStyle,

  /** 数据行样式：左对齐、带细边框 */
  dataRow: {
    alignment: { horizontal: 'left', vertical: 'center' },
    border: {
      top: { style: 'thin', color: 'D0D0D0' },
      bottom: { style: 'thin', color: 'D0D0D0' }
    }
  }
}
```

```
    } as CellStyle  
};
```

3.6 统一导出 API (index.ts)


```

// packages/excel-exporter/src/index.ts
import { WorkbookBuilder } from './workbook-builder';
import type { ExportOptions, ExportResult, SheetConfig } from './types';
import { wasmLoader } from './wasm-loader';

export * from './types';
export * from './style-presets';

/**
 * 导出 Excel (主入口)
 *
 * @example
 * ```ts
 * import { exportExcel, StylePresets } from '@your-org/excel-exporter';
 *
 * await exportExcel({
 *   sheets: [{
 *     name: '销售数据',
 *     columns: [
 *       { key: 'product', header: '产品名称', width: 20 },
 *       { key: 'revenue', header: '营收', width: 15, style:
StylePresets.currency }
 *     ],
 *     data: [{ product: 'Widget', revenue: 9999.99 }],
 *     freezeRows: 1
 *   }],
 *   filename: 'report.xlsx'
 * });
 * ```
 */
export async function exportExcel(options: ExportOptions):
Promise<ExportResult> {
  const startTime = performance.now();

  try {
    // 判断是否需要使用 Worker (数据量 > 5000 行时自动启用)
    const totalRows = options.sheets.reduce((sum, s) => sum +

```

```
s.data.length, 0);
    const useWorker = options.useWorker ?? totalRows > 5000;

    if (useWorker && typeof Worker !== 'undefined') {
        return exportInWorker(options);
    }

    // 主线程直接导出
    const builder = await WorkbookBuilder.create();
    options.sheets.forEach(sheet => builder.addSheet(sheet));
    const blob = await builder.toBlob();

    // 自动下载
    const filename = options.filename.endsWith('.xlsx')
        ? options.filename
        : `${options.filename}.xlsx`;
    const url = URL.createObjectURL(blob);
    const link = document.createElement('a');
    link.href = url;
    link.download = filename;
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
    URL.revokeObjectURL(url);

    return {
        success: true,
        blob,
        duration: performance.now() - startTime,
        rowCount: totalRows
    };
} catch (error) {
    return {
        success: false,
        error: error as Error,
        duration: performance.now() - startTime
    };
}
```

```
    }  
  }  
  
  /** Worker 模式导出（大数据量场景） */  
  async function exportInWorker(options: ExportOptions):  
    Promise<ExportResult> {  
    // 通过 Comlink 或原生 postMessage 与 Worker 通信  
    // 详见 3.7 节  
    // ...  
  }  
}
```

3.7 Web Worker 实现 (export.worker.ts)

```
// packages/excel-exporter/src/workers/export.worker.ts
import { initWasm, Workbook } from 'modern-xlsx';

self.addEventListener('message', async (event) => {
  const { sheets, filename } = event.data;

  try {
    await initWasm();
    const wb = new Workbook();

    sheets.forEach((sheet: any) => {
      const ws = wb.addSheet(sheet.name);
      // ... 构建逻辑（与主线程相同）
    });

    const buffer = await wb.toBuffer();
    self.postMessage({
      success: true,
      buffer,
      filename: filename.endsWith('.xlsx') ? filename :
`${filename}.xlsx`
    });
  } catch (error) {
    self.postMessage({ success: false, error: error.message });
  }
});
```

四、性能优化策略

4.1 批量写入优先

modern-xlsx 的 `batch().writeRows()` 比逐单元格写入快数倍。所有数据写入应优先使用批量 API。

4.2 WASM 懒加载

WASM 模块（1.1MB）在页面加载时不加载，仅在用户首次点击导出时加载。配合 `prefetch` 策略，在空闲时预加载：

```
// 在应用入口处，空闲时预加载
if ('requestIdleCallback' in window) {
  requestIdleCallback(() => {
    import('modern-xlsx').then(({ initWasm }) => initWasm());
  });
}
```

4.3 Worker 自动调度

- < 5000 行：主线程直接导出（Worker 启动成本 > 收益）
- ≥ 5000 行：自动切换到 Worker 线程
- ≥ 50000 行：Worker + 流式写入（如使用 turbo-xlsx 的流式能力）

4.4 降级策略

WASM 加载失败时，自动降级到 SheetJS（需预先安装作为备选）：

```
async function exportWithFallback(options: ExportOptions):
Promise<ExportResult> {
  try {
    await wasmLoader.ensureLoaded();
    return exportWithWasm(options);
  } catch {
    console.warn('WASM 加载失败，降级到 SheetJS');
    return exportWithSheetJS(options); // 备选方案
  }
}
```

五、实施计划与里程碑

阶段	周期	关键交付物	验收标准
Phase 1：技术预研	第 1-2 周	技术选型报告、Demo 验证	modern-xlsx 在 Vue3 + Vite 中跑通
Phase 2：核心开发	第 3-6 周	@your-org/excel-exporter 包	支持多 Sheet、样式、公式、冻结窗格

Phase 3: Worker 集成	第 7-8 周	Worker 多线程导出	1 万行导出不阻塞 UI
Phase 4: 首个 App 接入	第 9-10 周	admin-a 接入并上线	线上运行稳定，无报错
Phase 5: 全面推广	第 11-12 周	所有 App 接入	统一率 100%，沉淀文档

六、风险与应对

风险	概率	影响	应对措施
WASM 加载失败	中	高	预置 SheetJS 降级方案
浏览器不支持 WASM	低	高	检测后直接使用 SheetJS
Worker 中序列化开销大	中	中	仅对 >5000 行启用，阈值可调
modern-xlsx 版本更新不兼容	低	中	锁定版本，定期评估升级

七、挑战目标撰写（可直接使用）

挑战目标：基于 WebAssembly 构建 Monorepo 级高性能 Excel 导出引擎

背景与痛点：

项目采用 Monorepo 架构，包含 3 个以上后台管理应用，均存在大量数据导出需求。现有方案（SheetJS）存在明显性能瓶颈：导出 1 万行以上数据时页面卡顿，3 万行以上时频繁出现内存警告甚至崩溃。各应用导出逻辑重复，样式不统一，维护成本高。

挑战内容：

- WASM 核心引擎引入：**引入 modern-xlsx（Rust + WASM），替换现有 SheetJS 方案。根据官方 benchmark，批量写入 10 万行数据仅需 232ms，比 SheetJS 快 8.4 倍。
- Monorepo 共享包封装：**在 packages/excel-exporter 中封装统一的导出 API，包括多 Sheet 支持、单元格样式（字体/颜色/边框/对齐）、数字格式、公式、合并单元格、冻结窗格等。所有 App 通过 workspace:* 协议引用，实现一次开发、多处复用。
- Web Worker 多线程导出：**数据量超过 5000 行时自动切换至 Web Worker 线程执行，WASM 计算在 Worker 中运行，主线程仅负责进度展示和结果下载，彻底解决 UI 卡顿问题。
- 智能加载与降级策略：**WASM 模块（1.1MB）采用懒加载，仅在用户首次点击导出时加载；加载失败时自动降级至 SheetJS，保证功能可用性。
- 预设样式模板体系：**建立业务通用的样式预设（表头、金额、日期、百分比等），将导出代码从平均 50+ 行缩减至 **5 行以内**。

量化指标：

- 10 万行数据导出耗时从 ~2s 降至 **≤300ms**（提升 6 倍以上）
- 5 万行导出时主线程阻塞时间 **≤16ms**（用户无感知）
- 导出相关代码量减少 **70% 以上**（通过共享包复用）
- 内存溢出导致的导出失败率 **降至 0**
- 所有 App 接入共享包，**统一率 100%**

时间节点：

- 第 2 周：技术选型与 demo 验证
- 第 6 周：共享包核心功能开发完成
- 第 8 周：Web Worker 多线程方案落地
- 第 10 周：首个 App 接入并上线

- 第 12 周：所有 App 接入，完成效果评估